PICK ME: FOOLING UNCERTAINTY-BASED DATA SELECTION IN FAVOR OF YOUR DATA

Anonymous authors
Paper under double-blind review

ABSTRACT

Training modern deep learning models increasingly relies on automated data curation pipelines that select a small, high-value subset from massive raw corpora. Because full-dataset training is often computationally infeasible, practitioners depend on data selection strategies that aim to identify the most "informative" examples for model improvement. A common principle behind these methods is model uncertainty: examples on which a proxy model is uncertain are assumed to contain useful learning signals and are therefore preferentially included in the final training subset. In this work, we highlight a previously underexplored security vulnerability within these selection pipelines. We consider an adversary who can contribute or slightly modify a small portion of the candidate training data before subset selection takes place. Rather than directly attacking the final trained model, the adversary targets the data selection stage, crafting inputs that appear maximally informative under the selection criterion. We propose **Pick Me**, an attack that maximizes predictive entropy, and **Pick Me++**, a bilevel optimization approach that ensures adversarial samples remain high-entropy even as the proxy model learns. We empirically demonstrate in the vision domain that an attacker can systematically amplify the selection rate of their data, illustrating that uncertainty-driven data selection can inadvertently provide a lever for adversaries to steer the composition of curated training sets.

1 INTRODUCTION

Modern machine learning systems are increasingly trained on massive, heterogeneous datasets collected from the web, third-party providers, and large-scale data repositories. While such scale has enabled remarkable performance gains, it also introduces a fundamental bottleneck: training on the entire available dataset is often computationally infeasible. As a result, data curation pipelines have become a critical component of the training process, where a smaller, high-value subset of data is selected for final model training.

A widely adopted paradigm for data selection is to prioritize *informative* samples according to a proxy model. Among various criteria, *predictive uncertainty*, often measured via entropy, has emerged as a simple and effective heuristic (Coleman et al., 2020; Toneva et al., 2019). The intuition is that samples on which a model is uncertain are more likely to provide useful learning signal and therefore should be included in the training subset. This principle underlies a growing number of data selection and active learning methods across both vision and language domains.

However, these approaches implicitly assume that the candidate data pool is benign. In practice, this assumption is increasingly questionable. Modern data pipelines frequently ingest content from untrusted and decentralized sources, including web crawls, user-generated content, and data marketplaces. This raises a critical but largely unexplored question:

Can an adversary manipulate uncertainty-based data selection to control which data is chosen for training?

To illustrate the risk, consider the following scenario in the context of large language models (LLMs). Suppose an independent content creator strongly desires that their writing be included in the training corpus of a future LLM. Because they cannot influence the entire dataset, they instead

focus on manipulating their own documents. If the data curation pipeline relies on uncertainty-based selection, the creator can deliberately craft text that is difficult for a partially trained model to interpret, e.g., by mixing styles, introducing ambiguity, or using rare constructions. Such inputs are likely to induce high predictive entropy and therefore be considered "informative" by the selection mechanism. As a result, these samples are disproportionately likely to be included in the final training subset, allowing the creator to exert an outsized influence despite controlling only a small fraction of the data.

This example highlights a broader vulnerability: when uncertainty is used as a proxy for data value, it becomes a *targetable signal*. An adversary may not need to improve the quality of their data, and only need to make it *look informative* to the selection algorithm.

In this work, we formalize and study this threat. We consider an adversary who can inject or slightly modify a small subset of candidate training data prior to selection. Rather than attacking the final trained model directly, the adversary targets the *data selection stage*, crafting inputs that maximize their probability of being selected. This represents a shift in perspective from traditional data poisoning, which focuses on degrading model performance, to *selection manipulation*, where the goal is to control the composition of the final training dataset.

We propose two attack strategies. The first, **Pick-Me**, directly maximizes predictive entropy under a proxy model using projected gradient descent. While simple, this approach does not account for the retraining dynamics of the selection pipeline. To address this limitation, we introduce **Pick-Me++**, a bilevel optimization framework that explicitly models the training process used in data selection. Pick-Me++ constructs inputs that remain high-entropy even after the model is retrained, ensuring their selection.

We validate our approach on standard vision benchmarks, where continuous inputs enable controlled optimization. Our results show that even under small perturbation budgets, adversarial samples can achieve significantly higher selection rates compared to benign or random inputs. These findings demonstrate that uncertainty-based data selection, while effective for efficiency, introduces a new and previously underexplored attack surface.

Contributions. In summary, our contributions are as follows:

- We identify a novel vulnerability in uncertainty-based data selection pipelines and formalize a new threat model targeting the selection stage.
- We propose **Pick-Me**, an entropy-maximization attack that increases the likelihood of sample selection.
- We introduce **Pick-Me++**, a bilevel optimization method that accounts for retraining dynamics during selection.
- We empirically demonstrate that a small adversarial budget can yield disproportionate influence over the selected training subset.

Broader Implications for LLMs. While our empirical evaluation focuses on the computer vision domain due to the continuous nature of image data and the tractability of gradient-based optimization, the underlying vulnerability extends to any modality utilizing uncertainty-based selection. Modern LLMs heavily rely on similar automated curation pipelines to filter massive text corpora scraped from the web. Although directly optimizing discrete text tokens presents distinct optimization challenges compared to continuous image pixels, our findings in the vision domain serve as a critical proof-of-concept. By demonstrating that uncertainty signals are inherently manipulable, we highlight a severe, theoretical attack surface for LLM data curation, where adversaries might utilize discrete optimization or adversarial prompting to inject highly selected, toxic, or biased text into pre-training corpora.

2 Related Work

Data Subset Selection and Coresets. The goal of data subset selection is to reduce training costs while preserving model generalization by identifying the most informative training examples.

Foundational work in this area includes geometric approaches (Sener & Savarese, 2018), gradient-matching methods (Mirzasoleiman et al., 2020), and metrics based on prediction margin and uncertainty (Coleman et al., 2020). Recent studies have also explored "data diets," identifying examples that are easily forgotten or highly influential during the early stages of training (Toneva et al., 2019; Paul et al., 2021). While some works have extended these concepts to improve the efficiency of adversarial training (Dolatabadi et al., 2023), the underlying assumption in coreset literature is that the initial dataset, though perhaps noisy, is not actively adversarial. Our work shows that the very metrics used to define "informativeness" (such as entropy) can be actively exploited.

Data Poisoning and Backdoor Attacks. Data poisoning aims to manipulate a model's behavior by altering its training data. Classic backdoor attacks inject a specific trigger into a subset of training images, causing the model to misclassify any input containing that trigger at inference time (Gu et al., 2017). Clean-label poisoning attacks take this a step further by introducing imperceptible perturbations to training data without altering the ground-truth labels (Shafahi et al., 2018). Typically, these attacks assume the poisoned data will definitely be included in the training batch. In modern, massive-scale pipelines where aggressive filtering and subset selection occur, standard poisoned samples may simply be discarded as uninformative or anomalous. Our attack bridges this gap, acting as a "delivery mechanism" to ensure malicious samples survive the subset selection bottleneck.

Security of Data Pipelines and Valuation. Recent literature has begun exploring the security risks inherent in the supply chain of foundation models. For instance, Hu et al. (2025) highlights how data curation steps in LLMs introduce new attack surfaces. Concurrently, research on data valuation in federated learning demonstrates that participants can strategically manipulate their training behavior to inflate their estimated data value under Shapley-style metrics (Zheng et al., 2025). Unlike these settings, our work focuses on centralized pipelines and specifically targets the *uncertainty signal itself*. We formulate precise gradient-based optimization strategies to guarantee an attacker's inclusion in the final selected training subset.

Adversarial Examples and Robust Optimization. Adversarial examples were first systematically studied by Goodfellow et al. (2014), who showed that small, human-imperceptible perturbations can reliably fool deep neural networks and proposed the fast gradient sign method (FGSM) as an efficient way to construct such inputs. This line of work highlighted the inherent vulnerability of modern models. Building on this, Madry et al. (2018) formulated adversarial robustness as a principled min–max optimization problem, leading to the development of adversarial training methods that improve worst-case performance against bounded perturbations. While these works primarily focus on test-time robustness and defenses, our setting differs fundamentally: instead of degrading inference-time predictions, we exploit similar gradient-based mechanisms at the *data selection stage* to ensure malicious samples are preferentially included in the training subset.

3 PRELIMINARIES

Let $D = \{(x_i, y_i)\}_{i=1}^{|D|}$ denote a labeled dataset, where $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$. In many large-scale learning pipelines, training on the entire dataset D may be computationally expensive. A common alternative is to first select a small subset $S \subset D$, with $|S| \ll |D|$, and then train the final model only on S. The goal of coreset selection is to construct such a subset while preserving as much of the utility of the full dataset as possible.

Uncertainty-based coreset selection. One widely used strategy is to rank samples according to their predictive uncertainty under a proxy model trained on the data pool. Concretely, the victim company trains a model $f_{\theta^{(v)}}$ for E_{victim} epochs on the dataset D. For each sample x_i, the victim computes its predictive entropy

$$H_{\theta^{(v)}}(x_i) = -\sum_{c=1}^{C} P_{\theta^{(v)}}(y = c \mid x_i) \log(P_{\theta^{(v)}}(y = c \mid x_i)), \tag{1}$$

where C is the number of classes. The selected coreset S is then formed by choosing the top-$|S|$ samples with the largest entropy values. Intuitively, uncertainty-based methods prioritize samples on which the trained model is least confident, since such samples are often viewed as more informative.

Victim pipeline. In our setting, the victim company follows exactly this procedure: it first trains the victim model $f_{\theta^{(v)}}$ for E_{victim} epochs on the available data pool, and then selects the top-$|S|$ samples according to predictive entropy. These selected samples are subsequently used for the final training.

Threat model. We assume an adversary controls a small set of samples $M \subset D$, where $|M| \ll |D|$. Before the victim starts the coreset selection procedure, the adversary can insert these samples into the data pool or replace existing samples with them. For each attacker-controlled sample $x_i \in M$, the adversary is allowed to construct a perturbed version $\hat{x}_i$ subject to an ℓ_∞ perturbation budget, i.e., $\|\hat{x}_i - x_i\|_\infty \leq \epsilon$. The attacker's goal is to maximize the number of attacker-controlled samples that appear in the final selected subset S while minimally changing their desired samples (within ϵ).

Attacker knowledge. We assume the adversary knows the victim uses uncertainty-based coreset selection, but does not necessarily know the exact architecture of the victim model $f_{\theta^{(v)}}$ or the number of victim training epochs E_{victim}. To approximate the victim's behavior, the adversary trains its own surrogate model $f_{\theta^{(a)}}$, which we call the *attacker proxy model*, for E_{proxy} epochs. In general, $f_{\theta^{(a)}}$ may differ from $f_{\theta^{(v)}}$ both in architecture and in training duration. This captures a realistic black-box setting in which the attacker knows the selection principle but not the exact implementation details.

4 THE PICKME ATTACK

We now introduce PICKME, a simple attack that aims to make attacker-controlled samples appear highly informative under uncertainty-based coreset selection. The key idea is to perturb each attacker-controlled sample so that its predictive entropy under the attacker proxy model becomes as large as possible. If this high-entropy behavior transfers to the victim model, then those samples are more likely to be included in the victim's top-$|S|$ selection.

4.1 ATTACK OBJECTIVE

Given the attacker proxy model $f_{\theta^{(a)}}$, for each controlled sample $x_i \in M$, the adversary solves:

$$\hat{x}_i = \arg\max_{x \in \mathcal{X}} H_{\theta^{(a)}}(x) \quad \text{s.t.} \quad \|x - x_i\|_\infty \leq \epsilon. \tag{2}$$

This objective encourages the perturbed sample to induce a nearly uniform predictive distribution, thereby maximizing its uncertainty score.

4.2 OPTIMIZATION

The above problem is non-convex, so we approximately solve it using projected gradient descent (PGD). Starting from the original sample x_i, we iteratively update the input in the direction that increases entropy, and after each step project the perturbed sample back into the feasible ℓ_∞ ball around x_i.

More precisely, letting $\hat{x}_i^{(0)} = x_i$, the update at iteration t is

$$\hat{x}_i^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(x_i)} \left(\hat{x}_i^{(t)} + \alpha \cdot \text{sign} \left(\nabla_x H_{\theta^{(a)}}(\hat{x}_i^{(t)}) \right) \right), \tag{3}$$

where α is the step size, $\Pi_{\mathcal{B}_\epsilon(x_i)}$ denotes projection onto the ℓ_∞ ball of radius ϵ centered at x_i, and T denotes the total number of PGD steps. The final adversarial example is $\hat{x}_i^{(T)}$. Our method is also detailed in Algorithm 1.

PICKME is simple and computationally efficient. It only requires gradient access to the attacker proxy model and does not require knowledge of the victim model architecture or of E_{victim}. Moreover, the attack is performed entirely before the victim begins the coreset selection procedure.

At the same time, PICKME is not directly optimizing the actual victim objective. Instead, it maximizes entropy with respect to a surrogate model $f_{\theta^{(a)}}$, hoping that the resulting uncertainty transfers

Algorithm 1 PICKME: Entropy-Maximization Attack for Uncertainty-Based Coreset Selection

Require: Attacker-controlled set M, attacker proxy model $f_{\theta^{(a)}}$, attacker proxy epochs E_{proxy}, perturbation budget ϵ, step size α, number of steps T

Ensure: Perturbed attacker-controlled set $\hat{M}$

Initialize $\hat{M} \leftarrow \emptyset$
Train $f_{\theta^{(a)}}$ on the data pool D for E_{proxy} epochs.
for each $(x_i, y_i) \in M$ **do**
  Initialize $\hat{x}_i^{(0)} \leftarrow x_i$
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute predictive probabilities $p^{(t)} \leftarrow f_{\theta^{(a)}}(\hat{x}_i^{(t)})$
    Compute entropy $H(p^{(t)}) \leftarrow -\sum_{c=1}^{C} p_c^{(t)} \log p_c^{(t)}$
    Compute input gradient $g^{(t)} \leftarrow \nabla_x H(p^{(t)})$
    Take ascent step $\tilde{x}_i^{(t+1)} \leftarrow \hat{x}_i^{(t)} + \alpha \cdot \text{sign}(g^{(t)})$
    Project onto the ℓ_∞ ball around x_i: $\hat{x}_i^{(t+1)} \leftarrow \text{clip}(\tilde{x}_i^{(t+1)},\, x_i - \epsilon,\, x_i + \epsilon)$
  end for
  $\hat{M} \leftarrow \hat{M} \cup \{(\hat{x}_i^{(T)}, y_i)\}$
end for
return $\hat{M}$

to the victim model $f_{\theta^{(v)}}$. Its success therefore depends on the transferability of high-entropy adversarial examples across architectures and training regimes, which we also study empirically in our experiments.

5 EXPERIMENTS

We evaluate the effectiveness of PICKME across multiple datasets, model architectures, and attack settings. Our experiments aim to answer two key questions: (i) how effective the attack is when the attacker perfectly matches the victim pipeline, and (ii) how well the attack transfers when this assumption is relaxed.

5.1 EXPERIMENTAL SETUP

We conduct experiments on three standard image classification benchmarks: MNIST (LeCun et al., 1998), FashionMNIST (Xiao et al., 2017), and CIFAR-10 (Krizhevsky et al., 2009). We consider two model architectures: (i) a simple multi-layer perceptron (MLP), and (ii) ResNet18 (He et al., 2016). Both models are used as proxy models (attacker side) and victim models.

Attack setting. We assume the adversary controls a subset $M \subset D$ of the training pool. The adversary perturbs these samples using PICKME before the victim performs coreset selection. The victim then selects the top-$|S|$ samples based on predictive entropy.

Evaluation metric. For each experiment, we report the proportion of attacker-controlled samples that appear in the selected coreset:

$$\text{Selection Rate} = \frac{|\hat{M} \cap S|}{|M|} \times 100,$$

In the tables, each row corresponds to a fixed $|M|$, and each column corresponds to the coreset size $|S|$.

Baseline (No Attack). In the absence of an attack, selection is independent of M, and the expected proportion of selected attacker samples is simply $\frac{|S|}{|D|}$. We report this expected value as the baseline. Any improvement over this baseline indicates successful manipulation of the selection procedure.

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	100	0	100	0	100	0	100	100	100	100	100	100
PickMe-10	80	60	100	80	100	80	100	80	100	90	100	100
PickMe-100	69	59	87	83	90	86	93	91	100	94	100	98
PickMe-1000	35.7	31.4	76.6	60.3	89.3	73.4	94.7	84.6	98.7	91.6	100	97.3
PickMe-5000	9.18	8.8	31.44	26.24	47.94	40.94	67.62	59.48	93.04	78.22	99.44	93.76

Table 1: Proportion of attacker-controlled samples selected into the top-$|S|$ subset across MNIST and FashionMNIST for different coreset sizes. The total data pool size was $|D| = 60,000$ for both datasets. The attacker's proxy and victim's proxy models were both MLP.

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	0	0	0	0	0	0	0	0	100	0	100	100
PickMe-10	0	0	40	10	50	10	60	60	80	90	100	90
PickMe-100	8	5	34	20	53	31	75	48	94	77	98	95
PickMe-1000	3.2	3.9	18.3	14.2	34	24.3	58.6	38.1	84.5	67.7	94.7	90.5
PickMe-5000	2.66	2.8	13.26	9.4	25.86	16.68	46.4	28.48	73.78	54.84	89.14	78.04

Table 2: Proportion of attacker-controlled samples selected into the top-$|S|$ subset across MNIST and FashionMNIST for different coreset sizes. The total data pool size was $|D| = 60,000$ for both datasets. The attacker's proxy and victim's proxy models were both ResNet18.

5.2 Matched Proxy Setting

We first consider an idealized setting in which the attacker perfectly matches the victim pipeline, i.e., the proxy and victim models share the same architecture and are trained for the same number of epochs ($E_{\text{proxy}} = E_{\text{victim}}$).

Tables 1, 2, and 3 report the proportion of attacker-controlled samples that are selected into the top-$|S|$ subset across MNIST, FashionMNIST, and CIFAR-10. Recall that in the absence of an attack, the expected proportion is $\frac{|S|}{|D|}$, which is reflected in the "No Attack" rows.

Effectiveness of the attack. Across all datasets and architectures, PickMe consistently increases the selection rate of attacker-controlled samples compared to the baseline. Under the MLP setting (Table 1), the attack is extremely strong: even for moderate attacker budgets (e.g., $|M| = 100$), the attacker achieves over 90% selection once the coreset size reaches 10% on MNIST and Fashion-MNIST. In contrast, when using ResNet18 (Table 2), the attack remains effective but is noticeably weaker, especially for smaller coreset sizes. On CIFAR-10 (Table 3), we observe a similar trend: the attack significantly outperforms the baseline for both architectures, but achieves substantially higher selection rates under MLP than under ResNet.

Impact of model architecture. A key observation is that the attack is more effective for simpler models. Comparing Tables 1 and 2, ResNet consistently yields lower selection rates than MLP across all values of $|M|$ and $|S|$. For example, on MNIST with $|M| = 1000$ and $|S| = 20\%$, the selection rate drops from 94.7% (MLP) to 58.6% (ResNet). A similar gap is observed on CIFAR-10 (Table 3), where ResNet reduces the attack success but does not eliminate it. This suggests that stronger models provide some robustness, but uncertainty-based selection remains vulnerable.

Scaling with $|M|$. The effectiveness of the attack increases with the number of attacker-controlled samples. When $|M|$ is small (e.g., PickMe-1), the attack often fails for small coreset sizes, particularly under ResNet. However, even for modest values such as $|M| = 10$, the attacker quickly achieves high selection rates as $|S|$ increases. For larger attacker budgets (e.g., $|M| = 100$ or 1000),

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	0	0	100	0	100	0	100	0	100	0	100	0
PickMe-10	10	10	60	30	80	40	80	50	100	70	100	80
PickMe-100	18	3	35	10	47	27	56	40	79	62	96	79
PickMe-1000	16.3	3	47.6	13.4	63.6	22.9	76.8	37.6	91	58.5	98.1	74.1
PickMe-5000	4.92	2.4	19.28	10.92	32.36	19.88	53.8	34.44	80.12	56.5	92.94	74.62

Table 3: Proportion of attacker-controlled samples selected into the top-$|S|$ subset on CIFAR-10 for different coreset sizes. The total data pool size was $|D| = 50,000$. The attacker's proxy and victim's proxy models were the same.

Figure 1: Original samples $x \in M$ (top row) and adversarial samples $\hat{x} \in \hat{M}$ (bottom row) generated by PICKME with $\epsilon = 0.3$, $\alpha = 0.01$, and iterations $T = 100$. Each column corresponds to a different dataset (MNIST, CIFAR-10, and FashionMNIST).

the attack dominates the coreset across most settings. Interestingly, for very large $|M|$ (PickMe-5000), the selection rate may begin to decrease due to the fact that the model might start to learn all these high-entropy samples easier and make them low-entropy.

Summary. While CIFAR-10 is more challenging than MNIST and FashionMNIST, the qualitative behavior remains consistent. The attack reliably increases the likelihood of attacker-controlled samples being selected across all datasets. Although the magnitude of the effect varies with dataset complexity and model architecture, the vulnerability of uncertainty-based selection persists. Overall, these results demonstrate that even when using stronger architectures such as ResNet, uncertainty-based coreset selection remains highly vulnerable. While more expressive models can mitigate the attack to some extent, PICKME is still able to significantly bias the selection process, especially as the attacker budget or coreset size increases. We have also provided examples of the attack in Figure 1, which illustrates original samples and their adversarial counterparts after entropy maximization.

5.3 TRANSFERABILITY OF THE ATTACK

We now consider a more realistic setting where the attacker does not have exact knowledge of the victim's proxy model. First, we focus on *cross-architecture transfer*, where the attacker and victim use different model classes (MLP vs. ResNet18), while still assuming the same number of training epochs ($E_{\text{proxy}} = E_{\text{victim}}$). Tables 4, 5, and 6 report the results for MNIST, FashionMNIST, and CIFAR-10, respectively.

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	0	0	0	0	0	0	100	100	100	100	100	100
PickMe-10	0	20	30	30	40	50	90	60	100	90	100	100
PickMe-100	4	13	22	34	41	45	62	53	88	80	95	88
PickMe-1000	2.1	4.2	15.3	16.4	29.9	27.4	51	37.6	78	63.5	90.1	85.2
PickMe-5000	1.9	1.78	8.86	7.86	16.8	15.52	31.52	27	54.4	50.16	73.82	71.52

Table 4: Proportion of attacker-controlled samples selected into the top-$|S|$ subset across MNIST and FashionMNIST for different coreset sizes. The total data pool size was $|D| = 60,000$ for both datasets. The attacker's proxy model was MLP and victim's proxy model was ResNet18.

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST	MNIST	F-MNIST
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	0	0	0	0	0	0	0	0	0	0	100	0
PickMe-10	0	10	0	10	10	10	30	30	60	40	90	80
PickMe-100	5	3	16	13	28	18	51	31	78	53	91	81
PickMe-1000	2.4	2.8	12.9	11.3	23.8	18	42.1	31.3	67.5	54.4	83.9	72.5
PickMe-5000	2.14	2.14	11.12	8.76	22.02	16.82	37.48	29.76	65.56	51.4	83.18	69.84

Table 5: Proportion of attacker-controlled samples selected into the top-$|S|$ subset across MNIST and FashionMNIST for different coreset sizes. The total data pool size was $|D| = 60,000$ for both datasets. The attacker's proxy model was ResNet18 and victim's proxy model was MLP.

5.3.1 Cross-architecture transfer (still same epochs)

Across all datasets, PickMe remains effective even when the attacker and victim use different architectures. However, compared to the matched setting, the attack success is consistently reduced. For example, on MNIST (Table 4), PickMe-100 achieves 62% selection at 20% coreset size when attacking a ResNet victim using an MLP proxy, compared to over 90% in the matched MLP setting. A similar degradation is observed when reversing the roles (Table 5), indicating that the mismatch impacts both directions. An interesting observation is that transferability is not symmetric. Attacks generated using MLP proxies tend to transfer better to ResNet victims than vice versa. For instance, on FashionMNIST at $|S| = 40\%$ and $|M| = 100$, the MLP→ResNet setting achieves 80% selection, whereas the ResNet→MLP setting achieves only 53%. This suggests that simpler models may generate more transferable high-entropy perturbations. As in the matched setting, CIFAR-10 remains more challenging than MNIST and FashionMNIST. The gap between the attack and the baseline is smaller, and the degradation due to architecture mismatch is more pronounced. Nevertheless, the attack consistently outperforms the baseline across all configurations, demonstrating that transferability persists even in more complex settings.

5.3.2 Different Epochs and Different Architectures

In the experiments for previous section, we assumed that the attacker and victim use the same number of training epochs ($E_{\text{proxy}} = E_{\text{victim}}$). A natural next question is how the attack behaves when this assumption is also relaxed. We now consider a more realistic setting where the attacker does not have exact knowledge of anything. In particular, we study *joint transfer across both architecture and training dynamics*, allowing the attacker and victim to differ in both model class (MLP vs. ResNet18) and number of training epochs ($E_{\text{proxy}} \neq E_{\text{victim}}$).

Table 7 presents a transferability matrix for a representative setting ($|M| = 100$, coreset size 20%), where rows correspond to the attacker's proxy configuration and columns correspond to the victim's configuration.

Method / $\lvert M \rvert$	Coreset Size $\frac{\lvert S \rvert}{\lvert D \rvert}$											
	1%		5%		10%		20%		40%		60%	
	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet	MLP	ResNet
No Attack	1	1	5	5	10	10	20	20	40	40	60	60
PickMe-1	0	0	0	0	0	0	0	0	0	0	0	0
PickMe-10	10	0	10	0	30	0	50	0	70	20	70	70
PickMe-100	2	2	18	6	30	7	41	16	63	39	82	54
PickMe-1000	3.4	1.3	11.7	7.4	19.6	14	35.2	26.2	55.5	47.2	71.6	66.8
PickMe-5000	1.78	1.52	8.06	6.62	14.42	11.94	26.56	23.88	48.58	45.82	68.44	66.2

Table 6: Proportion of attacker-controlled samples selected into the top-$|S|$ subset on CIFAR-10 for different coreset sizes. The total data pool size was $|D| = 50,000$. Each column represents the attacker's proxy model and the victim's proxy is the other model in this experiment.

Attacker \ Victim	MLP (1 ep)	MLP (3 ep)	MLP (5 ep)	ResNet (1 ep)	ResNet (3 ep)	ResNet (5 ep)
MLP (1 ep)	88.0	70.0	57.0	31.0	36.0	33.0
MLP (3 ep)	63.0	72.0	88.0	28.0	42.0	36.0
MLP (5 ep)	47.0	79.0	70.0	30.0	36.0	41.0
ResNet (1 ep)	16.0	17.0	17.0	19.0	37.0	36.0
ResNet (3 ep)	17.0	14.0	22.0	36.0	41.0	43.0
ResNet (5 ep)	13.0	18.0	18.0	25.0	35.0	42.0

Table 7: Selection rate (%) for $|M| = 100$ and coreset size $= 20\%$ on CIFAR10. Rows correspond to attacker's proxy configuration; columns correspond to victim's proxy configuration.

Summary. Overall, these results show that PICKME does not rely on exact knowledge of the victim architecture or epochs. While cross-architecture mismatch reduces effectiveness, the attack still significantly biases the selection process. This highlights a fundamental vulnerability of uncertainty-based selection: high-entropy adversarial samples tend to transfer across models.

6 PICKME++ ATTACK

While PICKME maximizes entropy under a fixed attacker proxy model $f_{\theta^{(a)}}$, it does not explicitly account for the retraining step in the victim pipeline. In practice, the victim trains its model on the full dataset (which includes adversarial samples) before computing entropy scores. This retraining step may reduce the entropy of attacker's samples $\hat{M}$, and weaken the attack.

To address this limitation, we propose **PICKME++**, a bilevel optimization approach that explicitly models the victim's training procedure.

6.1 BILEVEL FORMULATION

For each attacker-controlled sample $(x_i, y_i) \in M$, the adversary solves:

$$\hat{x}_i = \arg \max_{\|x - x_i\|_\infty \le \epsilon} H_{\theta^*}(x) \tag{4}$$

$$\text{s.t.} \quad \theta^* = \arg\min_{\theta} \sum_{(x_k, y_k) \in D_i(x)} \mathcal{L}_{\text{CE}}(f_\theta(x_k), y_k), \tag{5}$$

where we define the modified dataset as:

$$D_i(x) = \big(D \setminus \{(x_i, y_i)\}\big) \cup \{(x, y_i)\}. \tag{6}$$

Here, θ^* represents the parameters obtained after training the model on the dataset in which x_i is replaced by the adversarial sample x.

Algorithm 2 PICKME++

Require: Original sample x_{orig}, label y, training loader $\mathcal{T}$, model constructor GETNEWMODEL(), perturbation budget ϵ, outer learning rate α, number of outer iterations T_{out}, number of inner epochs E_{in}, inner learning rate η

Ensure: Optimized adversarial sample x_{adv}

Initialize $x_{\text{adv}} \leftarrow x_{\text{orig}}$, $\quad x_{\text{adv}}$ requires gradient
Initialize outer optimizer on x_{adv} with learning rate α
for $t = 1, 2, \ldots, T_{\text{out}}$ **do**
  Sample a fresh model $f_{\theta^{(0)}} \leftarrow$ GETNEWMODEL()
  Initialize inner SGD optimizer with learning rate η
  Enter differentiable inner-loop context using `higher` library
  for $e = 1, 2, \ldots, E_{\text{in}}$ **do**
    Sample a random batch index $r_e \sim \text{Uniform}\{0, 1, \ldots, |\mathcal{T}| - 1\}$
    Set batch counter $j \leftarrow 0$
    for each mini-batch $B \in \mathcal{T}$ **do**
      if $j = r_e$ **then**
        Augment the current batch with the adversarial sample: $B \leftarrow B \cup \{(x_{\text{adv}}, y)\}$,
      end if
      Compute inner loss $\mathcal{L}_{\text{in}} \leftarrow \text{CE}(f_{\theta^{(j)}}(B_x), B_y)$
      Take one differentiable SGD step: $\theta^{(j+1)} \leftarrow \theta^{(j)} - \eta \nabla_\theta \mathcal{L}_{\text{in}}$
      Update batch counter $j \leftarrow j + 1$
    end for
  end for
  Compute probibility logits for x_{adv}: $p \leftarrow f_{\theta^*}(x_{\text{adv}})$
  Compute entropy $H(x_{adv}) \leftarrow -\sum_{c=1}^{C} p_c \log p_c$
  Define outer objective $\mathcal{L}_{\text{out}} \leftarrow -H(x_{\text{adv}})$
  Backpropagate through the unrolled inner optimization and update x_{adv} using Adam

$$x_{\text{adv}} \leftarrow \text{AdamStep}(x_{\text{adv}}, \nabla_{x_{\text{adv}}} \mathcal{L}_{\text{out}}, \alpha)$$

  Project x_{adv} back into the feasible set: $x_{\text{adv}} \leftarrow \text{clip}(x_{\text{adv}},\ x_{\text{orig}} - \epsilon,\ x_{\text{orig}} + \epsilon)$
end for
return final sample x_{adv}

6.2 INTERPRETATION

The inner optimization models the victim's training process: given a candidate adversarial sample x, the victim trains its model (for E_{victim} epochs) on the dataset $D_i(x)$, resulting in parameters θ^*. The outer optimization then adjusts x such that, after this training process, the entropy of the sample under the trained victim model is maximized.

6.3 DISCUSSION

PICKME++ explicitly accounts for the victim's retraining dynamics and therefore directly optimizes the true selection criterion used in the pipeline. As a result, it can produce adversarial samples that retain high entropy even after training, making them significantly more likely to be selected. However, this comes at a higher computational cost compared to PICKME, as it requires differentiating through the training process. Algorithm 2 illustrates our implementation of PickMe++.

7 EXPERIMENTS: PICKME++

Table 8 compares the ranking of attacker-controlled samples under No Attack, PICKME, and PICKME++. Overall, PICKME++ achieves better (lower) ranks than PICKME in most runs (3 out of 5), while showing comparable performance in one run and underperforming in another. This variability suggests that although modeling retraining dynamics can strengthen the attack, it also makes the optimization more challenging. In particular, PICKME++ is significantly more computationally intensive and exhibits instability during optimization, and may require more optimization steps or

Run	No Attack	PickMe	PickMe++
1	17982	463	207
2	13028	1852	784
3	19246	4518	1208
4	20251	271	1797
5	20127	1063	1190

Table 8: Rank of the perturbed sample after uncertainety selection across 5 runs, for No attack, PickMe, and PickMe++ scenarios. Ranks are computed among all 60,000 MNIST samples.

careful tuning to consistently outperform PICKME. Nevertheless, both methods substantially improve over the baseline, confirming that adversarial samples can reliably move toward the top of the selection ranking.

8 CONCLUSION

In this work, we identified a novel vulnerability in uncertainty-based data selection pipelines, showing that predictive uncertainty, which is commonly used as a proxy for informativeness, can be directly exploited by adversaries. We introduced PICKME, an entropy-maximization attack, and PICKME++, a bilevel extension that accounts for retraining dynamics during selection. Through extensive experiments, we demonstrated that even a small adversarial budget can significantly increase the likelihood that attacker-controlled samples are included in the selected training subset. While stronger models and mismatched settings reduce the effectiveness of the attack, they do not eliminate the underlying vulnerability.

Our findings highlight that the data selection stage itself is an important and previously underappreciated attack surface in modern machine learning pipelines. As large-scale systems increasingly rely on automated data curation from untrusted sources, ensuring the robustness of selection mechanisms becomes critical. Future work should explore defenses such as robust uncertainty estimation, diversity-aware selection, and more broadly reconsider the role of data curation as a security-sensitive component of the learning pipeline.

REFERENCES

Cody Coleman, Christopher Yeh, Stephen Mussmann, Baharan Mirzasoleiman, Peter Bailis, Percy Liang, Jure Leskovec, and Matei Zaharia. Selection via proxy: Efficient data selection for deep learning. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HJg2b0VYDr`.

Hadi M Dolatabadi, Sarah M Erfani, and Christopher Leckie. Adversarial coreset selection for efficient robust training. *International Journal of Computer Vision*, 131(12):3307–3331, 2023.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Qiang Hu, Xiaofei Xie, Sen Chen, Lili Quan, and Lei Ma. Large language model supply chain: Open problems from the security perspective. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 169–173, 2025.

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rJzIBfZAb.

Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. Coresets for data-efficient training of machine learning models. In *International Conference on Machine Learning*, pp. 6950–6960. PMLR, 2020.

Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. Deep learning on a data diet: Finding important examples early in training. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, 2021. URL https://openreview.net/forum?id=Uj7pF-D-YvT.

Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=H1aIuk-RW.

Ali Shafahi, W Ronny Huang, Mahyar Najibi, Octavian Suciu, Christoph Studer, Tudor Dumitras, and Tom Goldstein. Poison frogs! targeted clean-label poisoning attacks on neural networks. *Advances in neural information processing systems*, 31, 2018.

Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=BJlxm30cKm.

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

Shuyuan Zheng, Sudong Cai, Chuan Xiao, Yang Cao, Jianbin Qin, Masatoshi Yoshikawa, and Makoto Onizuka. Data overvaluation attack and truthful data valuation in federated learning. *arXiv preprint arXiv:2502.00494*, 2025.